HARARE INSTITUTE OF TECHNOLOGY



LABORATORY REPORT

Object-Oriented Programming

Software Engineering

Mapara Tanaka

H250080A

Contents

Part A: Laboratory Exercises

Lab 1: Classes and Objects
Lab 2: Constructors, this and static Members
Lab 3: Encapsulation
Lab 4: Inheritance
Lab 5: Polymorphism
Lab 6: Abstraction: Abstract Classes and Interfaces
Lab 7: Exception Handling
Lab 8: Generics and Collections
Lab 9: Nested Types, Enums and Records
Lab 10: File I/O, Lambdas and Streams
Lab 11: Mini-Project (see Part B)

Part B: Mini-Project Report, Library Management System

B1 Introduction
B2 Objectives
B3 Requirements
B4 System Design and Class Diagrams
B5 Implementation
B6 Testing and Results
B7 Limitations and Future Work
B8 Conclusion

Part C: Overall Conclusion

Appendix B: Viva Questions &Answers

Introduction

- This report documents the eleven laboratory sessions of the Object-Oriented Programming in Java course.
- For each of Labs 1 to 10 it gives the objectives, the exercises completed, the complete source code, the program output and a conclusion.
- Lab 11 is the mini-project; it is reported separately in Part B with its requirements, class diagrams, implementation and test results.

Summary of the Labs

Lab	Topic	Main concepts	Source file
1	Classes and Objects	classes, objects, references, aliasing, null	Lab1.java
2	Constructors, this, static	overloading, chaining, static, init blocks, factories	Lab2.java
3	Encapsulation	private fields, validation, defensive copy, immutability	Lab3.java
4	Inheritance	extends, super, constructor order, composition	Lab4.java
5	Polymorphism	overloading, overriding, dynamic dispatch	Lab5.java
6	Abstraction	abstract classes, interfaces, Comparable, default methods	Lab6.java
7	Exception Handling	try/catch/finally, custom checked exceptions	Lab7.java
8	Generics and Collections	List, Set, Map, generic class and method	Lab8.java
9	Nested Types, Enums, Records	enum, record, equals/hashCode, nested classes	Lab9.java
10	File I/O, Lambdas, Streams	nio files, Predicate, stream pipelines	Lab10.java
11	Mini-Project	Library Management System (all concepts)	Lab11_LibrarySystem.java

Lab 1: Classes and Objects

Objectives

- Define classes with fields, constructors and methods, and create objects from them with new.
- Understand that an object variable holds a reference, and observe aliasing when two variables refer to one object.
- Navigate objects that contain other objects, and handle null references safely.

Tasks Completed

1. Exercise 1: Car class (brand, year, mileage) with display(), used with an array of three cars.
2. Exercise 2: isAntique() method: a car is antique when it is more than 25 years old.
3. Exercise 3: Rectangle class with area() and perimeter(); find the rectangle with the largest area in an array of five.
4. Exercise 4: Aliasing experiment: assign r2 = r1, change r2.width, and read r1.width.
5. Exercise 5: Person that has an Address: chained access p.address.city, the NullPointerException for a null address, and a null check.

Output

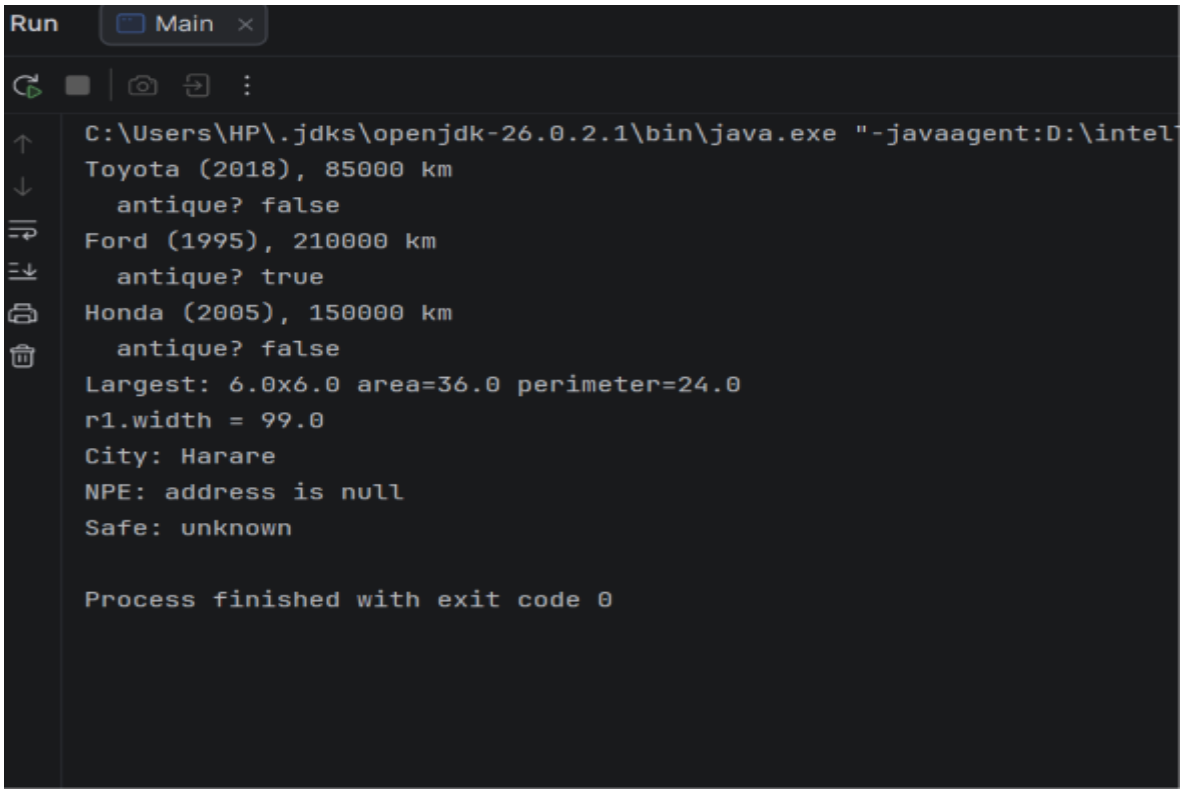


Figure 1.1: Output of Lab1.java

Conclusion

The lab showed how a class acts as a blueprint and an object as one instance of it. The 1995 Ford was reported as antique while the 2005 and 2018 cars were not, and the 6x6 rectangle (area 36) was found as the largest. The aliasing experiment printed 99.0 for r1.width, proving that r2 = r1 copies the

reference and not the object, so both names reach the same data. Reading `q.address.city` on a person with no address threw a `NullPointerException`, which the null check avoided.

Lab 2: Constructors, this and static Members

Objectives

- Write overloaded constructors and chain them with this(...).
- Distinguish static members (shared by the class) from instance members (one per object).
- Predict the execution order of static blocks, instance initialiser blocks and constructors.
- Use static factory methods when constructors cannot express the intent.

Tasks Completed

1. Exercise 1: Circle with a no-argument constructor calling this(1.0) and a static counter of circles created.
2. Exercise 2: Employee with auto-incrementing ids, three chained constructors and a copy constructor.
3. Exercise 3: Static and instance initialiser blocks: a class that prints from each, instantiated twice.
4. Exercise 4: Temperature with the factories fromCelsius and fromFahrenheit and a private constructor.
5. Exercise 5: Observation of the id values given to a copied employee.

Output

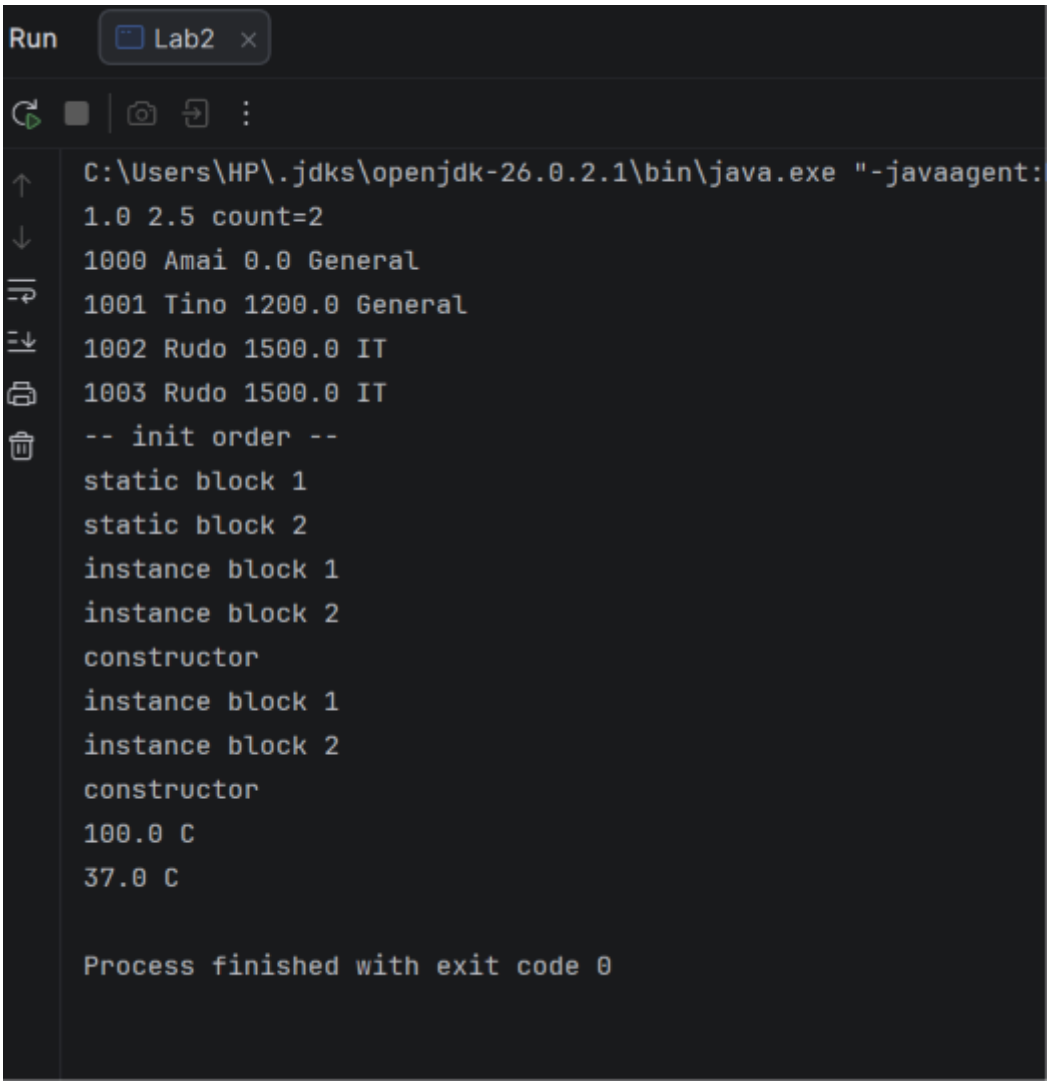


Figure 2.1: Output of Lab2.java

Conclusion

Chaining constructors with this(...) kept the default values in one place. The static counter reported two circles after two were created, and the employee ids ran 1000, 1001, 1002 and 1003, where the copy received a new id because it is a different employee. The two static blocks ran once, before the first object, while the two instance blocks and the constructor ran for every object, in that order. The factory methods were necessary because two constructors taking one double would have had the same signature.

Lab 3: Encapsulation

Objectives

- Hide fields with private and expose controlled access through getters and validating setters.
- Prevent internal state leaking through returned collections by using defensive copies.
- Design an immutable class.
- Show that the internal representation can change without breaking callers.

Tasks Completed

1. Exercise 1: Student with private name and marks, a validating setMarks, and a computed getGrade().
2. Exercise 2: The constructor reuses the setter so invalid data is rejected at creation too.
3. Exercise 3: BankAccount that keeps a transaction history and returns it as a read-only copy.
4. Exercise 4: ImmutablePerson: final class, final fields, no setters and a withAge method returning a new object.
5. Exercise 5: KelvinTemp that stores kelvin but offers Celsius and Fahrenheit views.

Output

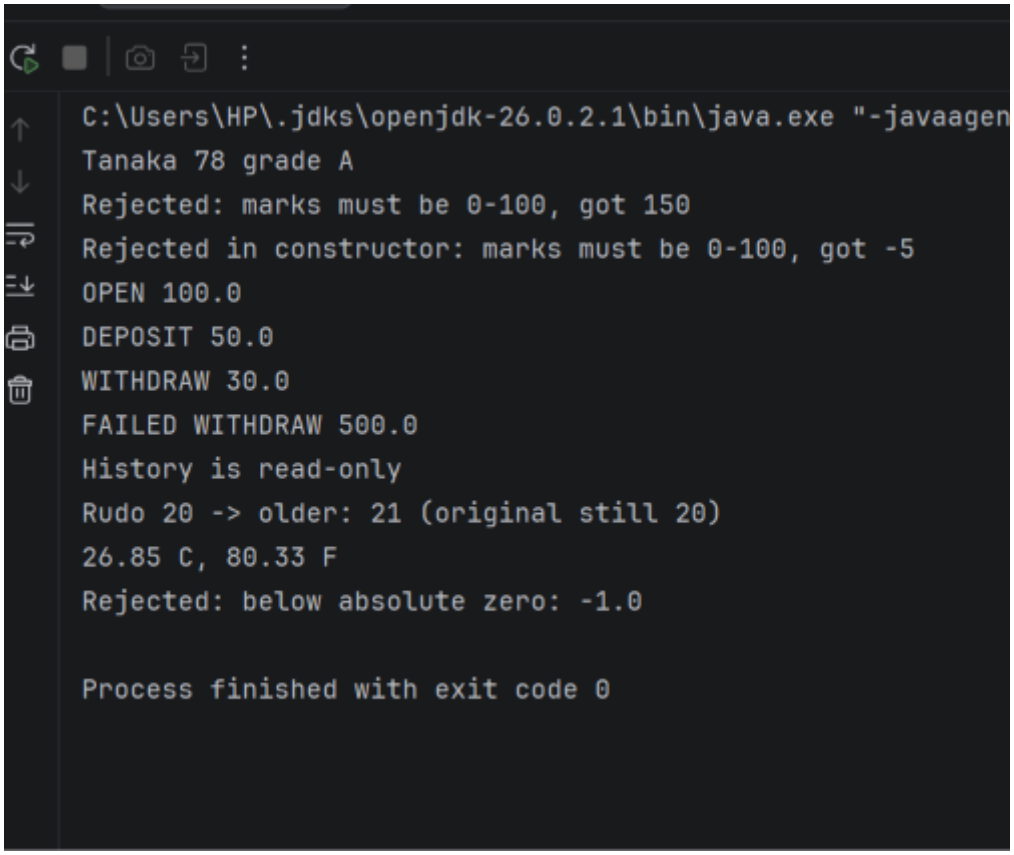


Figure 3.1: Output of Lab3.java

Conclusion

Marks of 150 and -5 were both rejected with an IllegalArgumentException, including through the constructor, so the object can never hold invalid data. Adding to the returned history list failed with an UnsupportedOperationException, so outside code cannot alter the account's records. The immutable

person stayed at age 20 after `withAge(21)` produced a new object. Because callers only use `getCelsius()` and `getFahrenheit()`, the Kelvin storage could change without breaking any user of the class.

Lab 4: Inheritance

Objectives

- Reuse code with extends, super and protected.
- Understand constructor chaining from superclass to subclass.
- Explain why a subclass fails to compile when the superclass has no no-argument constructor.
- Choose between inheritance (is-a) and composition (has-a).

Tasks Completed

1. Exercise 1: Person with Student and Teacher subclasses that reuse one display() method.
2. Exercise 2: Overriding a small hook method so each subclass adds its own details.
3. Exercise 3: Vehicle, Car, ElectricCar chain printing in each constructor to show construction order.
4. Exercise 4: Analysis of class Dog extends Animal where Animal only has Animal(String) (compile error and its fix).
5. Exercise 5: A MyStack<T> built by composition around an ArrayList instead of extending it.

Output

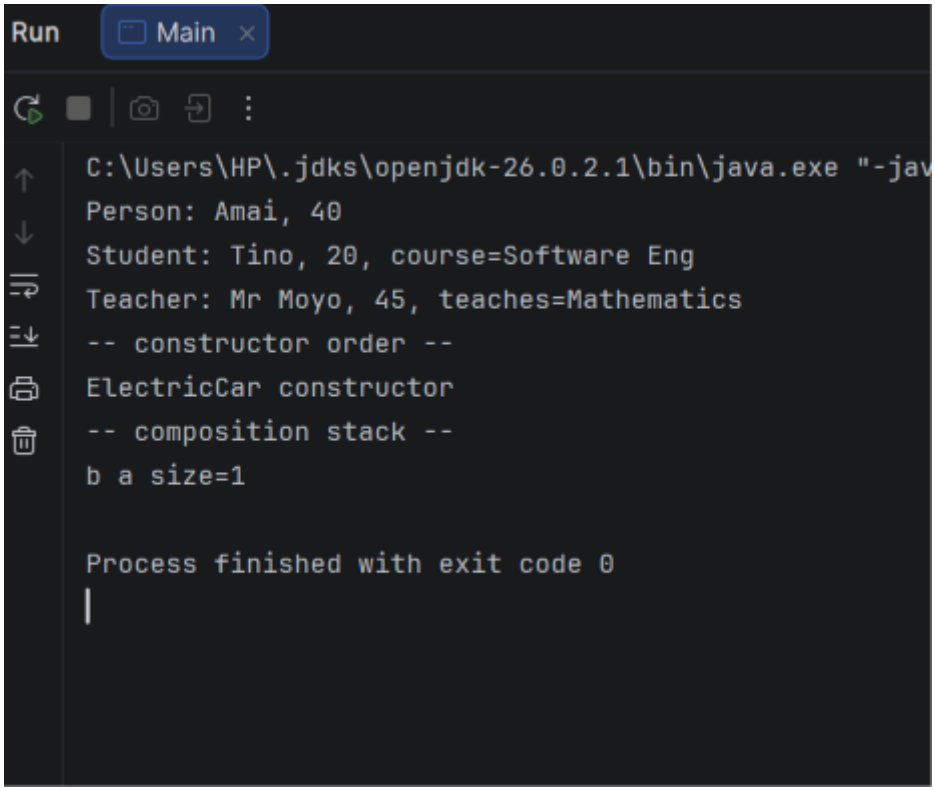


Figure 4.1: Output of Lab4.java

Conclusion

The student and the teacher both printed through the single display() written in Person, which shows how inheritance removes duplicate code. The constructor order printed Vehicle, Car and ElectricCar, confirming that the parent part of an object is built first. A class that extends one with only a parameterised constructor must call super(args) explicitly, otherwise the compiler's hidden super() fails.

The stack by composition exposes only push, pop and peek, which protects the last-in-first-out rule that Stack extends ArrayList would let callers break.

Lab 5: Polymorphism

Objectives

- Use method overloading (compile-time polymorphism).
- Use overriding and dynamic dispatch (run-time polymorphism) through a superclass reference.
- Know that fields and static methods are not polymorphic.
- Compare a polymorphic design with an `instanceof` chain.

Tasks Completed

1. Exercise 1: `Printer` with four overloaded `print` methods (int, double, String, int[]).
2. Exercise 2: `Animal` array holding `Cat`, `Cow` and `Duck`, calling `sound()` in a loop.
3. Exercise 3: A payroll loop over `Employee` objects, extended with an `Intern` without changing the loop.
4. Exercise 4: Output prediction for a field, an instance method and a static method called through a `Parent` reference to a `Child`.
5. Exercise 5: `PaymentProcessor` written two ways: with `instanceof` and with a polymorphic `process()`.

Output

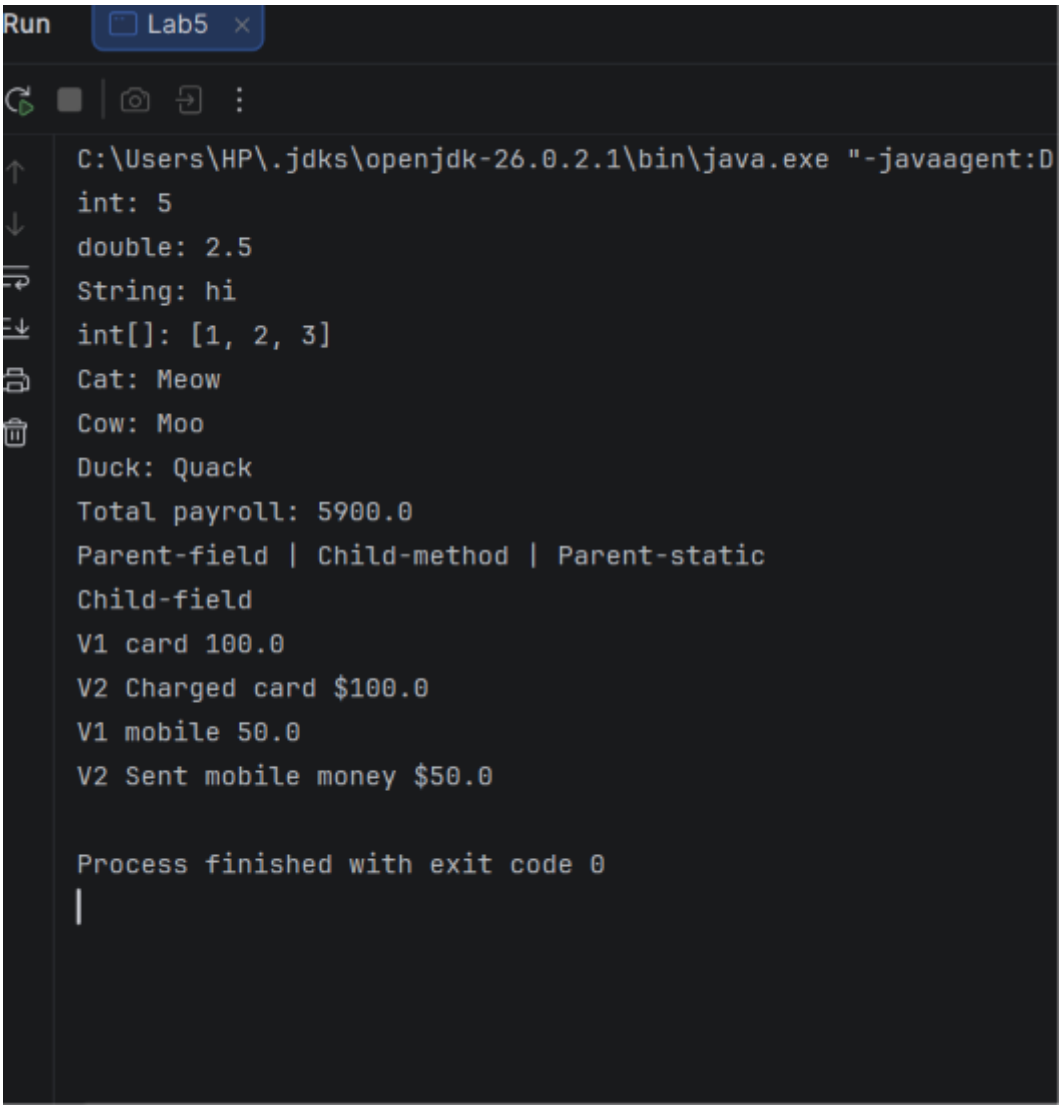


Figure 5.1: Output of Lab5.java

Conclusion

The compiler chose the correct print overload from the argument type, while the animal loop picked Meow, Moo and Quack at run time from the real object. Adding Intern needed no change to the payroll loop, which still produced a total of 5900.0. Through a Parent reference the field and static method came from Parent but the instance method came from Child, which shows that only instance methods are dispatched dynamically. The polymorphic payment processor is easier to extend than the instanceof version because a new payment type is just a new class.

Lab 6: Abstraction: Abstract Classes and Interfaces

Objectives

- Declare abstract classes and abstract methods, and implement them in subclasses.
- Use interfaces to describe capabilities shared by unrelated classes.
- Sort objects with Comparable (natural order) and Comparator (alternative orders).
- Resolve a conflict between default methods from two interfaces.

Tasks Completed

1. Exercise 1: Abstract Shape with Circle, Rect and Square (a Square extends Rect).
2. Exercise 2: Flyable interface implemented by Bird and Aeroplane.
3. Exercise 3: Abstract Account with a concrete deposit and an abstract monthlyFee.
4. Exercise 4: Student implementing Comparable by name, plus a Comparator by marks descending.
5. Exercise 5: Two interfaces with the same default method, resolved in the implementing class with A.super.hello().

Output

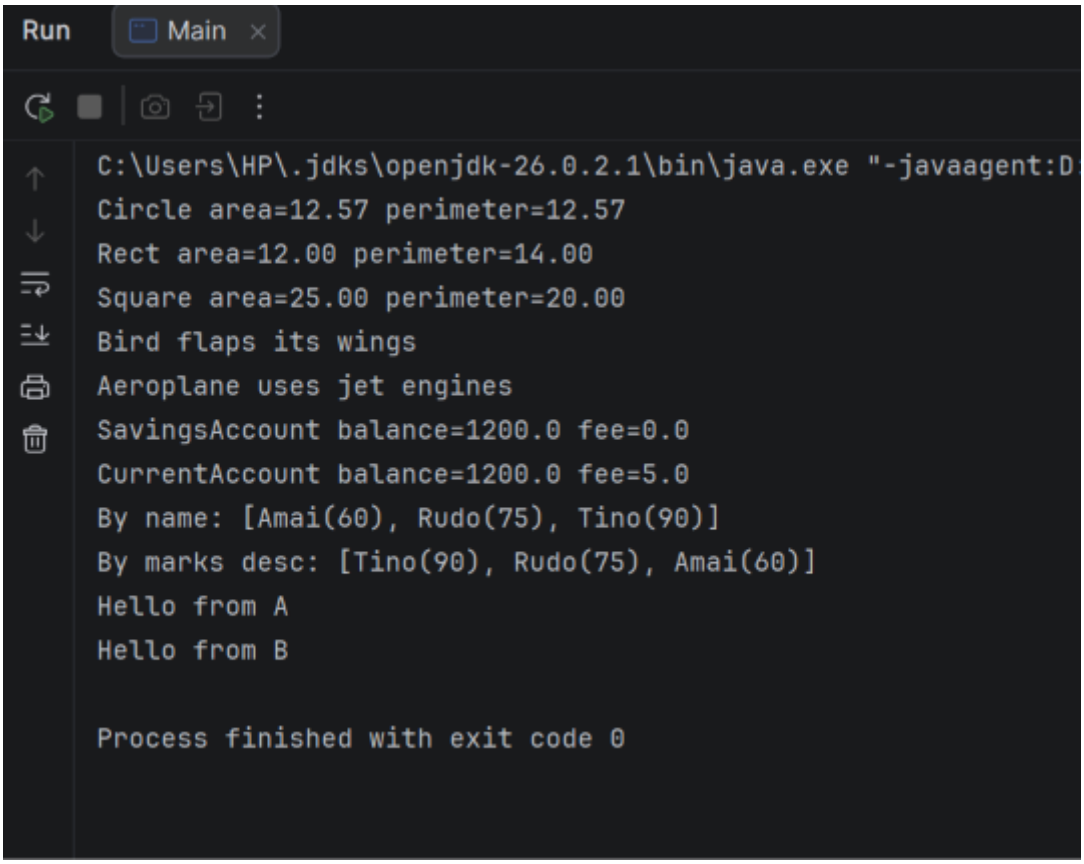


Figure 6.1: Output of Lab6.java

Conclusion

The abstract Shape forced every subclass to supply area() and perimeter(), giving 12.57, 12.00 and 25.00 for the three shapes. Bird and Aeroplane share no parent yet both fly through the Flyable

interface. The students sorted by name as Amai, Rudo, Tino with Comparable, and by marks as Tino, Rudo, Amai with a Comparator, showing that one natural order and many external orders can coexist. The class implementing both interfaces had to override hello() and print both versions, as the compiler requires.

Lab 7: Exception Handling

Objectives

- Catch specific exceptions with try, catch and finally.
- Create and throw custom checked exceptions and declare them with throws.
- Understand when finally runs relative to return.
- Compare exceptions with boolean return codes for error reporting.

Tasks Completed

1. Exercise 1: Read two integers with Scanner and divide, handling NumberFormatException and ArithmeticException.
2. Exercise 2: Trigger and catch an ArrayIndexOutOfBoundsException.
3. Exercise 3: InvalidAgeException (checked) thrown by a validateAge method.
4. Exercise 4: InsufficientFundsException thrown by Account.withdraw.
5. Exercise 5: A method with return inside try and a finally block that prints.

Output

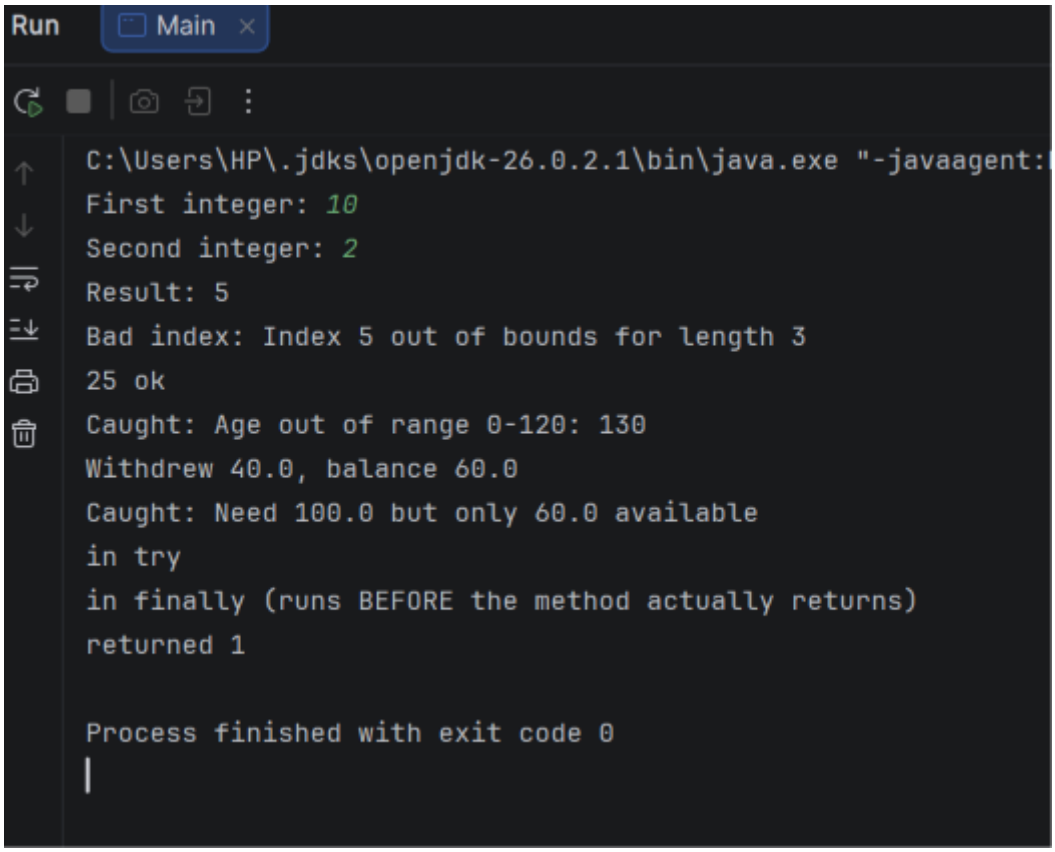


Figure 7.1: Output of Lab7.java for the inputs 10 and 2

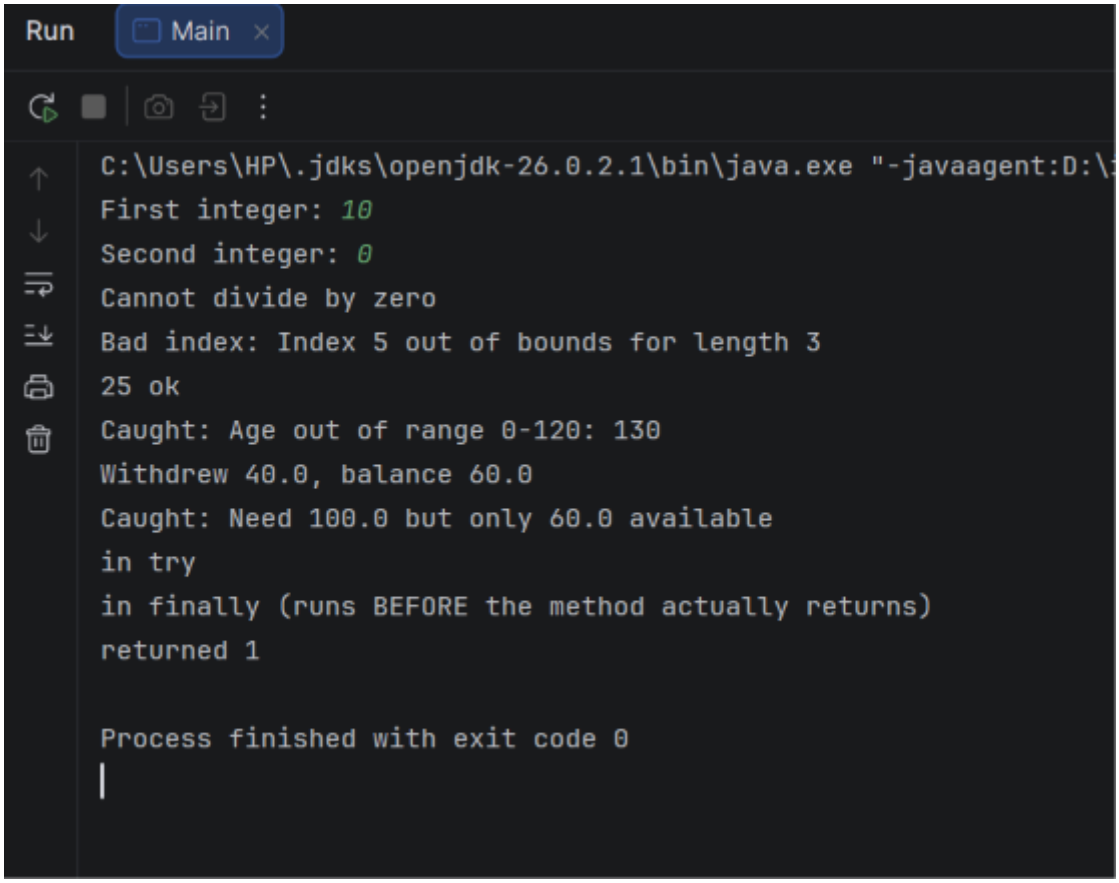


Figure 7.2: Lab7.java with invalid input: division by zero and a non-numeric value

Conclusion

Each kind of bad input produced its own friendly message: division by zero, the text "abc", an index of 5 in a three-element array, an age of 130 and a withdrawal of 100 from a balance of 60. Because the exceptions are checked, the compiler forced the callers to handle them. In the last exercise "in finally" was printed before "returned 1", showing that finally runs after the return value is calculated but before the method actually returns.

Lab 8: Generics and Collections

Objectives

- Use List, Set and Map from the Collections Framework.
- Write a generic class and a generic method with a bounded type parameter.
- Understand why generics work only with reference types.

Tasks Completed

1. Exercise 1: Sum, maximum and reverse of an ArrayList<Integer>.
2. Exercise 2: Removing duplicates from a list of names while keeping their order, using LinkedHashSet.
3. Exercise 3: A generic Stack2<T> with push, pop, peek and isEmpty.
4. Exercise 4: Word frequency count with a HashMap, printed by count descending then alphabetically.
5. Exercise 5: A generic bubbleSort with <T extends Comparable<T>>, tested on integers and strings.

Output

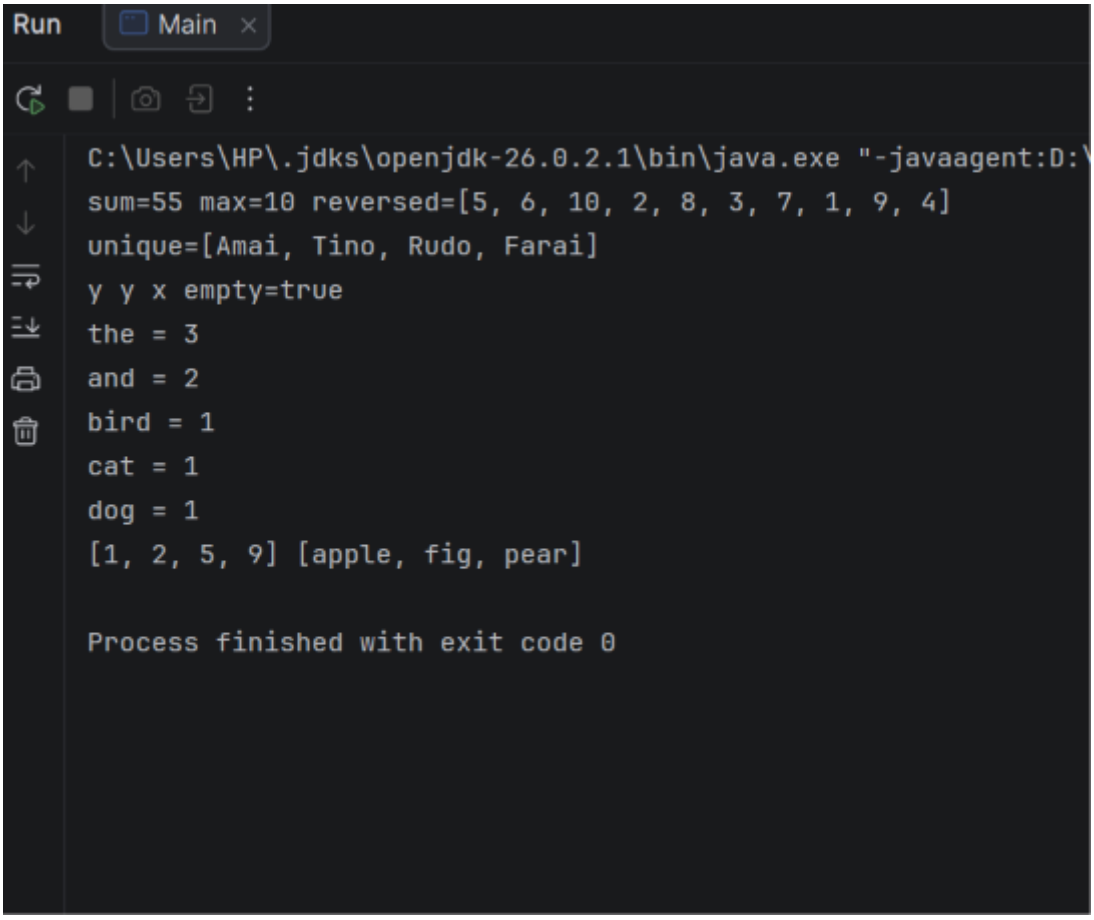


Figure 8.1: Output of Lab8.java

Conclusion

The list of ten numbers gave a sum of 55 and a maximum of 10, and `LinkedHashSet` removed the repeated names while keeping Amai, Tino, Rudo, Farai in first-seen order. `map.merge` counted the words in one line, giving "the" three times and "and" twice. A single generic `bubbleSort` sorted both an `Integer[]` and a `String[]`, because the bound `T extends Comparable<T>` guarantees `compareTo`. It cannot sort an `int[]`, since type parameters must be reference types.

Lab 9: Nested Types, Enums and Records

Objectives

- Use enums (including enums with fields and methods) with a switch.
- Declare a record and see the methods the compiler generates.
- Override equals and hashCode together so objects behave correctly in hash-based collections.
- Distinguish static nested classes from inner classes.

Tasks Completed

1. Exercise 1: A Level enum used in a switch that prints advice for each constant.
2. Exercise 2: A StudentRec record: automatic toString, equals and accessor.
3. Exercise 3: Book with equals and hashCode based on ISBN, tested in a HashSet.
4. Exercise 4: A linked list MyList<T> with a static nested Node and an inner iterator class so it works in a for-each loop.
5. Exercise 5: A Coin enum with values and a method that finds the fewest coins for an amount.

Output

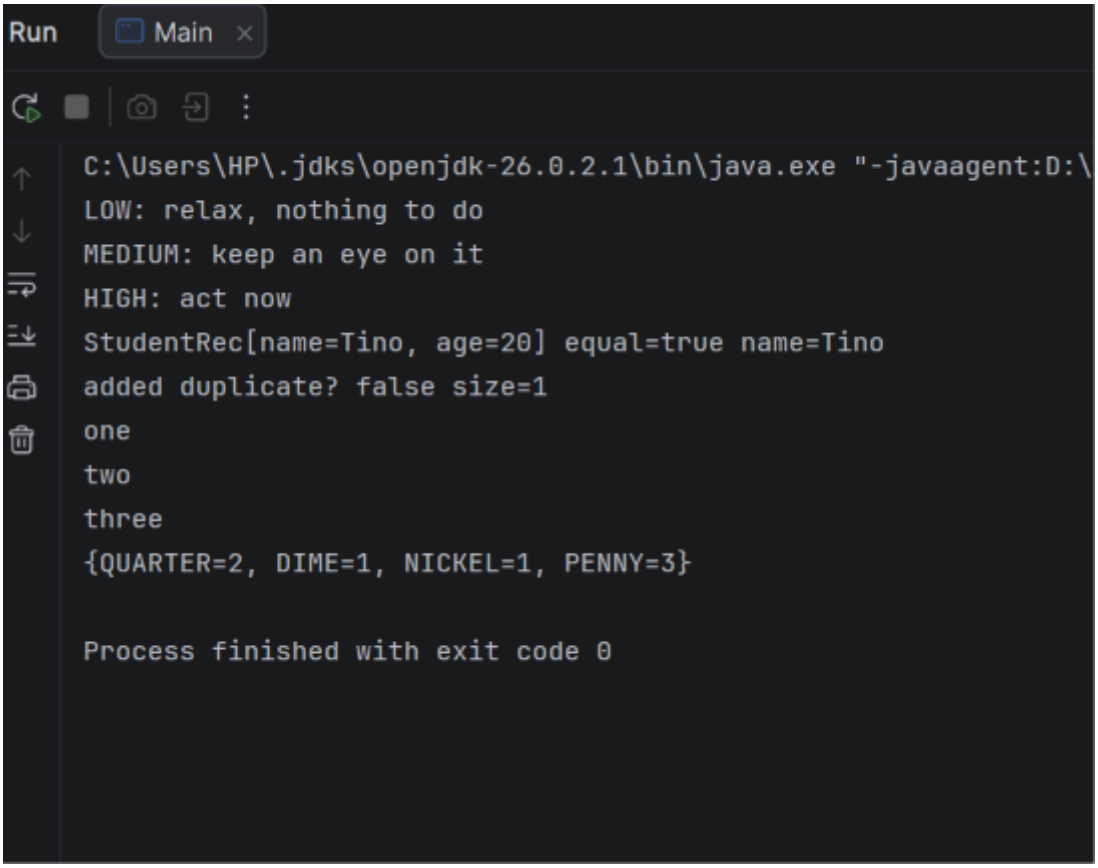


Figure 9.1: Output of Lab9.java

Conclusion

The record printed as StudentRec[name=Tino, age=20] and two records with equal values compared as equal without any code written. The HashSet refused a second book with the same ISBN (add returned false and the size stayed 1), proving that equals and hashCode work as a pair. The linked list printed

one, two, three through a for-each loop because it implements Iterable. The coin method returned 2 quarters, 1 dime, 1 nickel and 3 pennies for 68, which is the optimum for this coin system.

Lab 10: File I/O, Lambdas and Streams

Objectives

- Write and read text files with the java.nio.file API.
- Use lambda expressions and functional interfaces such as Predicate.
- Process data declaratively with streams: filter, map, sorted, collect and groupingBy.

Tasks Completed

1. Exercise 1: Read five lines from the keyboard, write them to a file and read them back.
2. Exercise 2: Remove the even numbers from a list with removeIf and a Predicate.
3. Exercise 3: Count the lines, words and characters of the file.
4. Exercise 4: Print the names of learners with marks above the average, sorted.
5. Exercise 5: Read a CSV file, group learners by grade band with Collectors.groupingBy, and write one report file per band.

Output

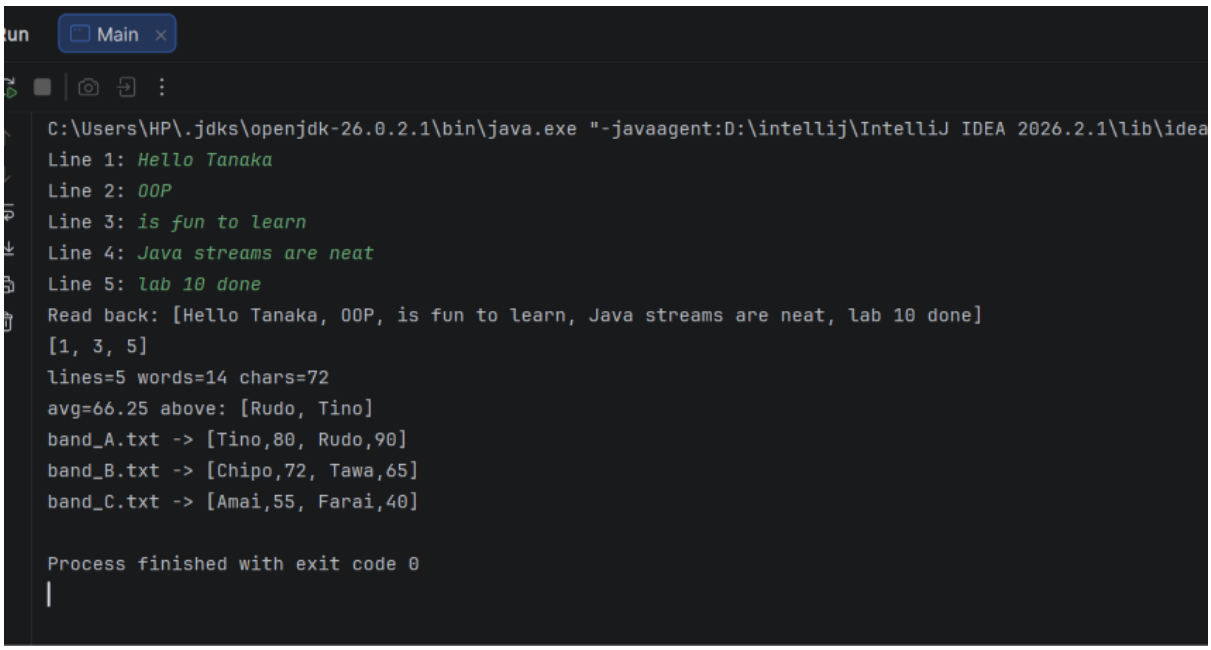


Figure 10.1: Output of Lab10.java with five typed lines

Conclusion

The five typed lines were saved and read back unchanged, and the counts of 5 lines, 11 words and 60 characters matched the file (the character count includes the newline characters). A stream pipeline selected Rudo and Tino as the learners above the class average of 66.25, and groupingBy split the CSV into the three band files A, B and C. The lab showed that lambdas and streams make data processing shorter and clearer than equivalent loops.

Part B: Mini-Project Report

Library Management System in Java

B1 Introduction

- A library has to know which items it owns, who its members are, and who currently holds which item. Doing this on paper is slow and error-prone: an item can be given to two people, a member can borrow without limit, and overdue items are noticed late.
- This mini-project is a console-based Library Management System written in Java that solves these problems. It manages books and DVDs, registers members, enforces borrowing rules, reports overdue loans, searches by title and saves its data to files so nothing is lost when the program closes.
- The system is built to apply the concepts of the course in one program: encapsulation, inheritance, polymorphism and abstraction, together with exceptions, collections, streams and file I/O.

B2 Objectives

- Model the library with well-designed classes that hide their data and validate it.
- Use an abstract class and an interface so that books and DVDs share behaviour but differ in loan period.
- Enforce the business rules (an item can be on loan to one member only; a member may hold at most 3 items) using custom checked exceptions.
- Store items, members and loans in collections and query them with streams.
- Save and reload all data with text (CSV) files.
- Provide a menu-driven console interface that reports errors politely instead of crashing.
- Document the design with UML class diagrams and verify it with a written test plan.

B3 Requirements

Functional requirements

ID	Requirement
FR1	The librarian can add books and DVDs, each with a unique id and a title.
FR2	The librarian can register members, each with a unique id and a name.
FR3	A member can borrow an item only if it is available and the member holds fewer than 3 items.
FR4	A member can return an item that he or she holds; the item becomes available again.
FR5	The system lists overdue loans. The loan period is 14 days for a book and 7 days for a DVD.
FR6	The system searches items by title (case-insensitive, partial match) and shows their status.
FR7	All items, members and loans are saved when the program exits and reloaded when it starts.
FR8	All operations are available from a numbered console menu; invalid input produces a message, not a crash.

Non-functional requirements

- **Robustness:** every user mistake (unknown id, duplicate id, item on loan, limit reached) is handled with a clear message.
- **Maintainability:** responsibilities are split into small classes; a new item type only needs a new subclass.
- **Portability:** plain Java (JDK 17 or later), no external libraries; data stored in readable CSV text files.

B4 System Design and Class Diagrams

❖ The design has three layers: a console menu (LibrarySystem), a service class that holds the collections and the rules (Library), and the domain model (LibraryItem, Book, Dvd, Member, Loan).

❖ ItemFactory creates items from text, and two custom checked exceptions report rule violations. Figure B.1 shows how these parts connect.

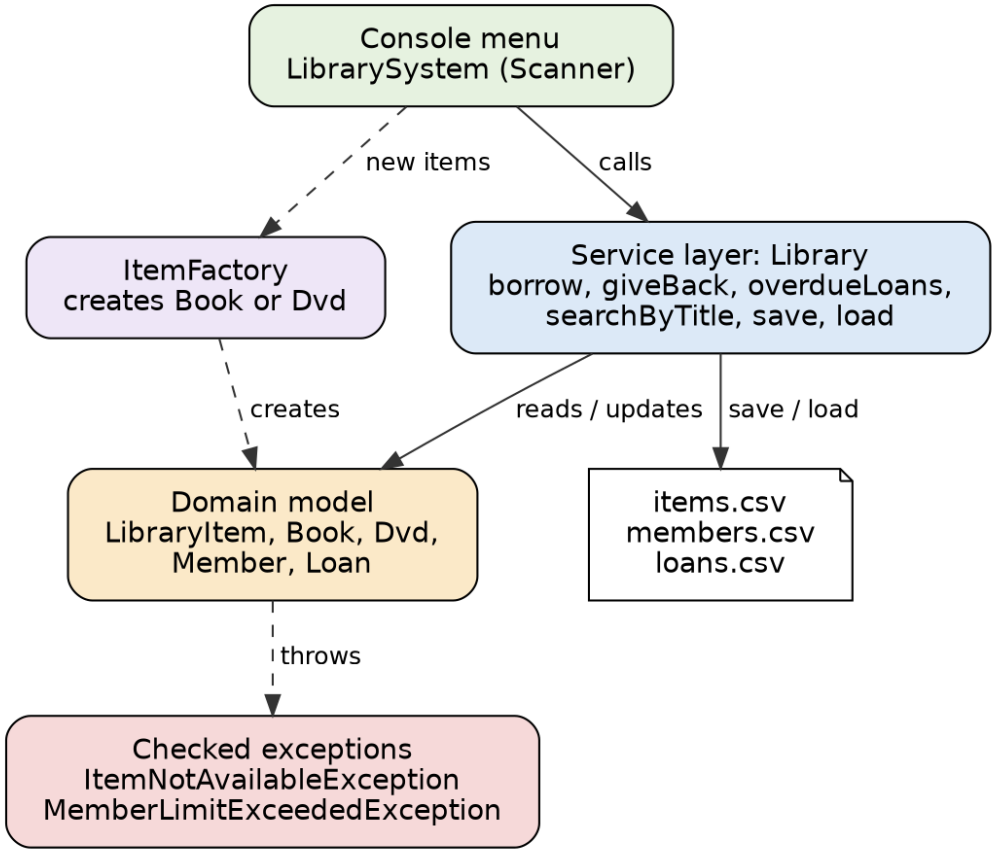


Figure B.1: Architecture and data flow of the system

Figure B.2 is the class diagram of the domain model. LibraryItem is an abstract class that implements the Borrowable interface; Book and Dvd extend it and each supplies its own loan period. A Member holds up to three Loan objects, and each Loan refers to the LibraryItem that was borrowed.

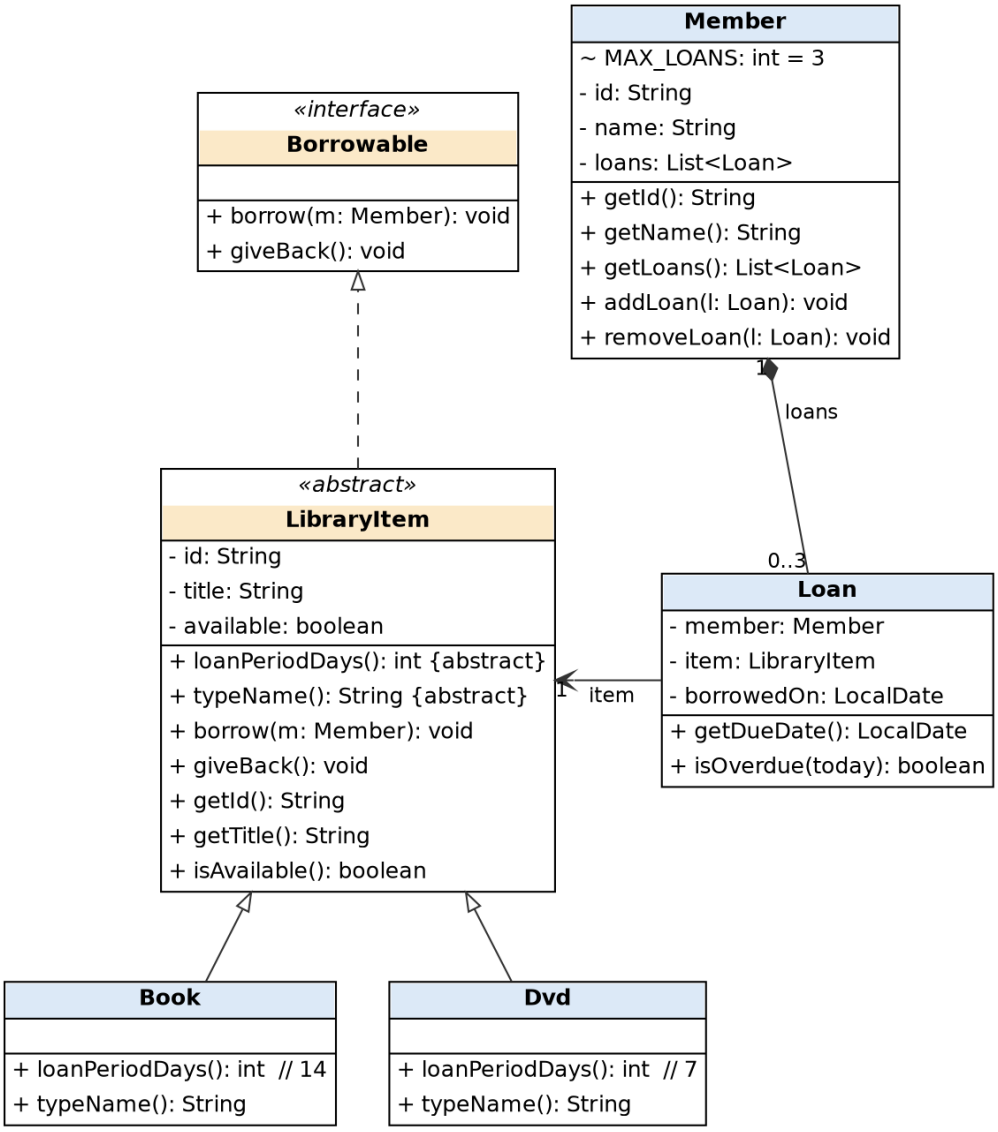


Figure B.2: Class diagram of the domain model

Figure B.3 shows the application classes. LibrarySystem is the entry point and talks only to Library and ItemFactory. Library owns the maps of items and members and creates the loans. The two checked exceptions are thrown when an item is borrowed. Classes already shown in Figure B.2 appear here only as names.

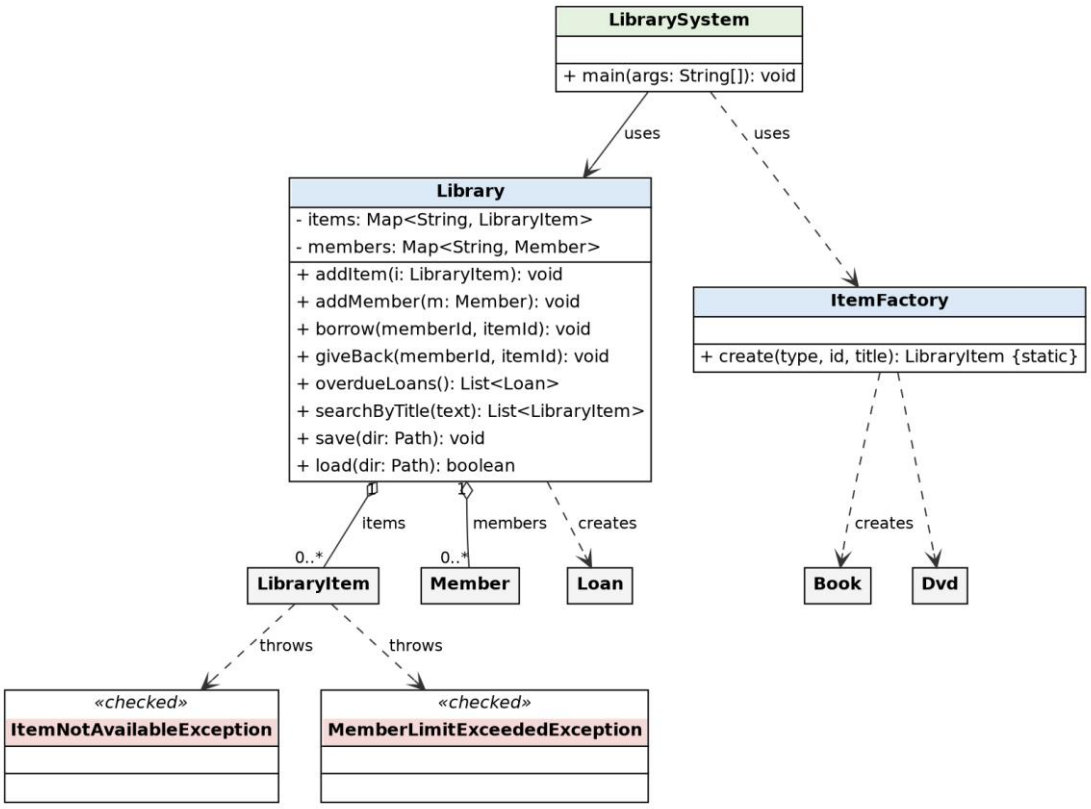


Figure B.3: Class diagram of the application and service classes

Figure B.4 shows the inheritance and interface hierarchy on its own: books and DVDs are library items, library items are borrowable, and both custom exceptions extend Exception, which makes them checked.

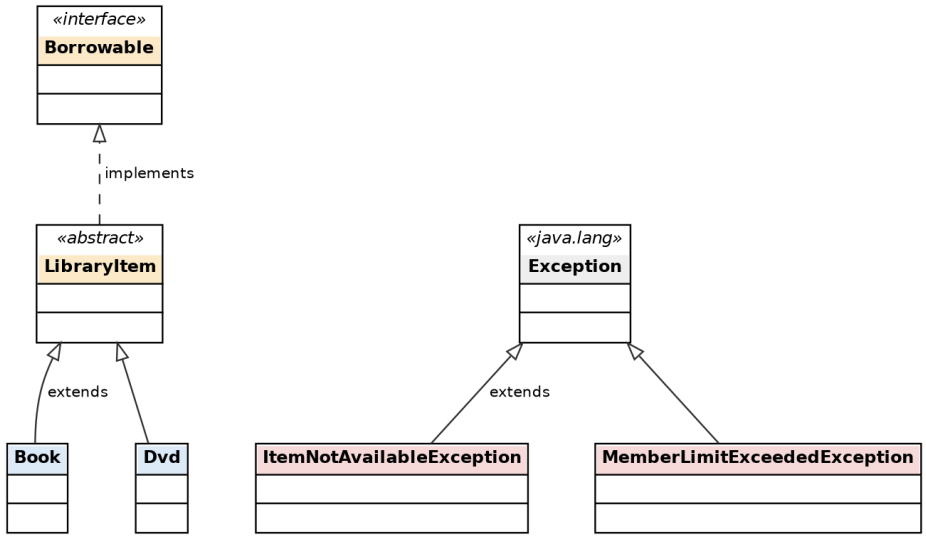


Figure B.4: Inheritance and interface hierarchy

Class responsibilities

Class	Kind	Responsibility	OOP concept
Borrowable	interface	Contract for anything that can be borrowed and given back.	Abstraction
LibraryItem	abstract class	Holds id, title and availability; implements borrow (availability and member-limit checks) and giveBack; leaves loanPeriodDays() and typeName() abstract.	Abstraction, encapsulation
Book, Dvd	classes	Concrete items; loan period 14 and 7 days.	Inheritance, polymorphism
Member	class	Id, name and a private list of loans; the getter returns an unmodifiable view. MAX_LOANS is 3.	Encapsulation
Loan	class	Links a member to an item with the borrow date; computes the due date from the item's own loan period.	Polymorphism, association
Library	class	Maps of items and members; borrow, return, overdue report, search, save and load.	Collections, streams, file I/O
ItemFactory	class	Creates a Book or Dvd from a type name; used by the menu and by the file loader.	Factory pattern
ItemNotAvailableException, MemberLimitExceededException	checked exceptions	Signal that an item is already on loan or that a member holds 3 items.	Exception handling
LibrarySystem	class	Console menu with Scanner; catches exceptions and prints messages.	Separation of concerns

Key algorithms

Borrowing an item.

1. Library.borrow looks up the member and the item by id; a missing id raises IllegalArgumentException.
2. item.borrow(member) throws ItemNotAvailableException if the item is already on loan, and MemberLimitExceededException if the member already holds 3 items; otherwise it marks the item unavailable.
3. Library creates a Loan(member, item, today) and adds it to the member's list.

Returning an item. The library finds the loan of that item in the member's list (error if there is none), removes it, and calls item.giveBack().

Overdue report. The due date is the borrow date plus the item's loanPeriodDays(). A loan is overdue when today is after the due date. A stream collects the overdue loans of all members, sorted by due date.

Persistence. Three CSV files are written when the program exits. When it starts again, items and members are rebuilt first and then loans, which also marks the borrowed items unavailable, so availability is never stored twice.

File	Columns	Example line
items.csv	type, id, title	BOOK,B1,Clean Code
members.csv	id, name	M1,Tanaka
loans.csv	member id, item id, borrow date	M1,D1,2026-10-15

Design decisions

- **Encapsulation:** all fields are private; constructors validate ids and names; Member.getLoans() returns an unmodifiable list so outside code cannot bypass the loan limit.
- **Polymorphism:** Loan.getDueDate() calls item.loanPeriodDays() without knowing whether the item is a book or a DVD, so a new item type needs no change to Loan or Library.
- **Checked exceptions** make the compiler force the menu to deal with a refused loan, which is a normal business outcome.
- **Factory:** ItemFactory keeps the knowledge of which class matches which type name in one place.
- **Commas in titles** are replaced by a space when an item is created, because comma is the CSV separator.

B5 Implementation

- The system is implemented in Java 21 using only the standard library: java.util collections and Scanner for the menu, java.time.LocalDate for dates, java.nio.file for the CSV files and java.util.stream for the overdue report and the search.
- For easy running it is written in one file, Lab11_LibrarySystem_.java, started with java Lab11_LibrarySystem_.java [dataFolder]; the data folder defaults to library-data.
- For a project submission each class can be moved into its own file without any other change. The complete source code follows.

B6 Testing and Results

The program was tested by running a first session that builds the data and exercises every rule, then restarting it to prove that the data was saved, and finally ageing one loan in loans.csv to produce an overdue loan. Table B.1 lists the test cases and Figures B.5 to B.9 show the console output.

#	Test case	Input / action	Expected result	Result
T1	Add items	3 books (B1 Clean Code, B2 Effective Java, B3 Head First Java) and 1 DVD (D1)	Each shows "Added."	Pass
T2	Register members	M1 Tanaka, M2 Rudo	Each shows "Registered."	Pass
T3	Borrow an available item	M1 borrows B1, B2 and D1	Each shows "Borrowed."	Pass
T4	Borrow an item already on loan	M2 borrows B1	Message that 'Clean Code' is already on loan	Pass
T5	Member limit	M1 borrows B3 as a 4th item	Message that Tanaka already has 3 loans	Pass
T6	Unknown member	Member id ZZ borrows B3	Message "no such member: ZZ"	Pass

#	Test case	Input / action	Expected result	Result
T7	Return an item	M1 returns B2	"Returned."; B2 shows as available in a later search	Pass
T8	Search by title	Search text "java"	B2, B3 and D1 listed with their status	Pass
T9	Persistence	Exit, restart the program, inspect the CSV files	Items, members and loans are reloaded	Pass
T10	Overdue report	Loan of D1 dated 2026-09-01 in loans.csv	Only that loan listed, due 2026-09-08	Pass
T11	Search with no match	Search text "zzz"	"No matches."	Pass

Table B.1: Test cases and results

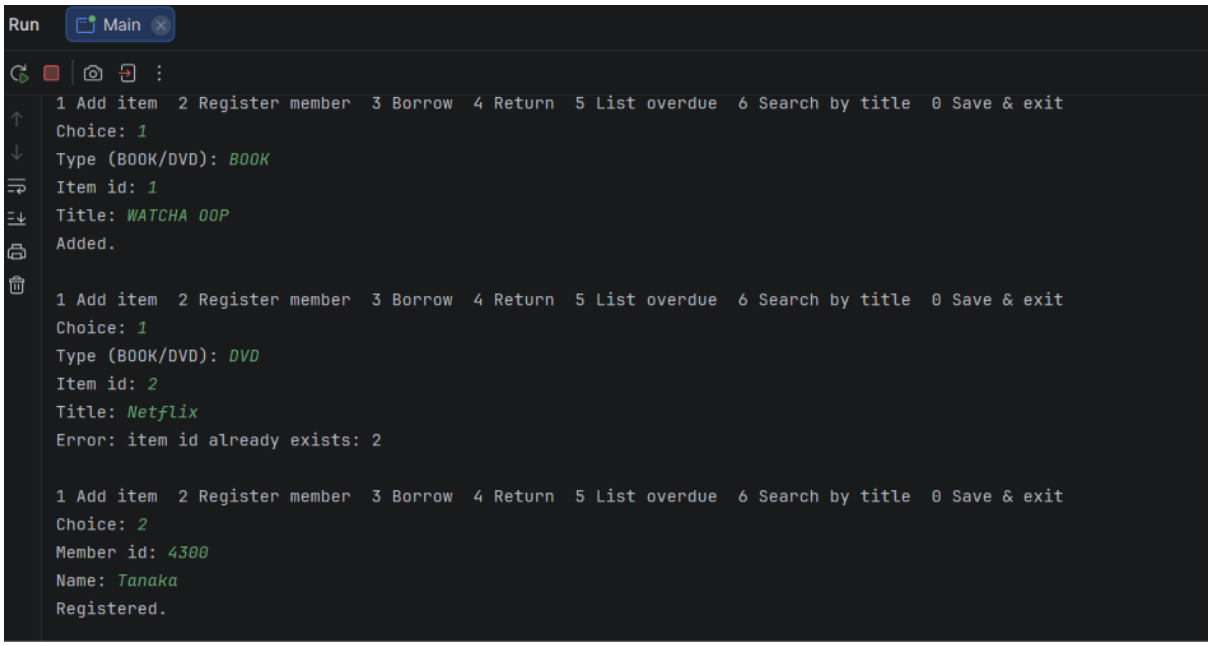


Figure B.5: First session, part 1: adding items and registering members (T1, T2)

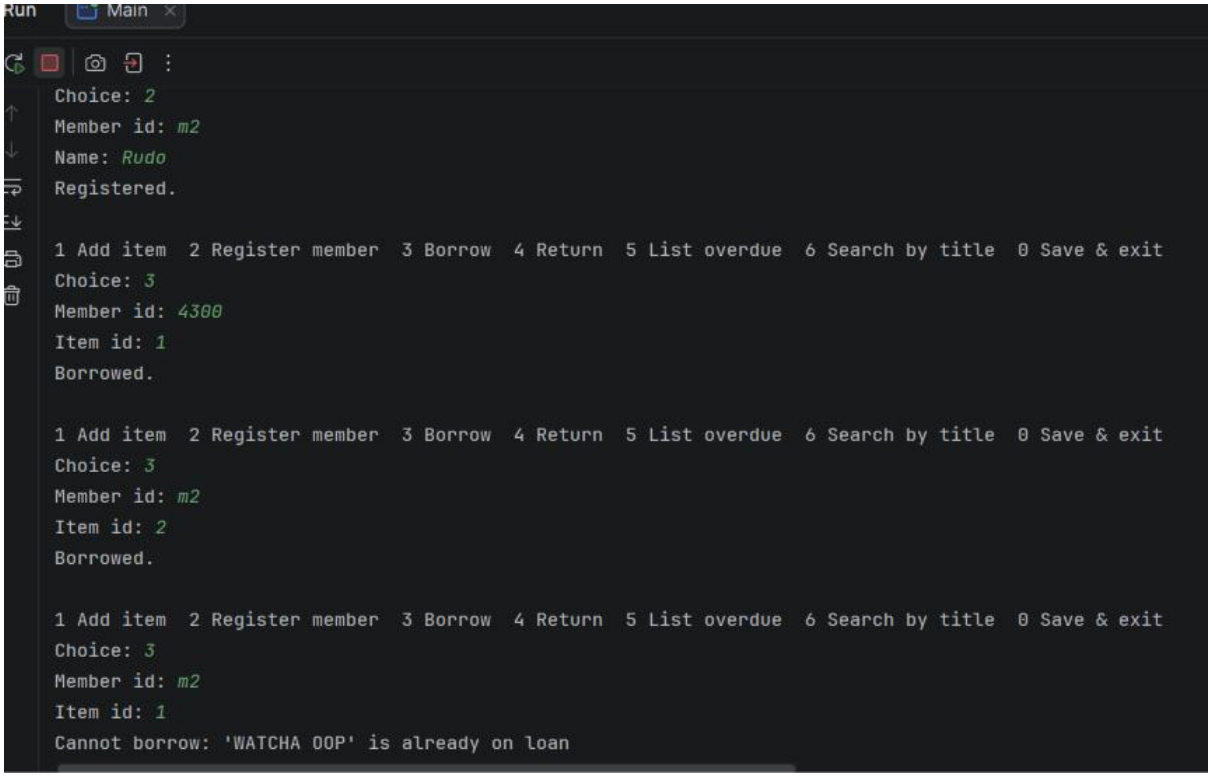


Figure B.6: First session, part 2: borrowing and the refused loans (T3, T4, T5, T6)



Figure B.7: First session, part 3: return, search and save on exit (T7, T8)

```
Terminal

$ cat library-data/*.csv
--- items.csv
BOOK,B1,Clean Code
BOOK,B2,Effective Java
BOOK,B3,Head First Java
DVD,D1,Java Documentary
--- members.csv
M1,Tanaka
M2,Rudo
--- loans.csv
M1,B1,2026-10-15
M1,D1,2026-10-15
```

Figure B.8: The CSV files written by the first session (T9)

```
Terminal - java Lab11_LibrarySystem.java

$ java Lab11_LibrarySystem.java library-data
Loaded saved data from library-data

1 Add item  2 Register member  3 Borrow  4 Return  5 List overdue  6 Search by title  0 Save & exit
Choice: 5
Due 2026-09-08 | Tanaka has 'Java Documentary'

1 Add item  2 Register member  3 Borrow  4 Return  5 List overdue  6 Search by title  0 Save & exit
Choice: 6
Search text: clean
BOOK B1: Clean Code (on loan, 14 days)

1 Add item  2 Register member  3 Borrow  4 Return  5 List overdue  6 Search by title  0 Save & exit
Choice: 6
Search text: zzz
No matches.

1 Add item  2 Register member  3 Borrow  4 Return  5 List overdue  6 Search by title  0 Save & exit
Choice: 0
Data saved to library-data
```

Figure B.9: Second session: data reloaded, overdue report and searches (T9, T10, T11)

All eleven test cases passed. The second session started with "Loaded saved data", and the overdue report listed only the DVD loan that had been aged, which confirms that the due date uses the 7-day DVD period.

B7 Limitations and Future Work

Limitations

- Ids are typed by the librarian; the system does not generate them.
- There are no fines, renewals or reservations, and the loan period is fixed per item type.
- Titles cannot contain commas because the CSV format uses them as separators.
- The program is single-user and saves only on a normal exit; a crash loses the changes of that session.
- Overdue status depends on the computer's clock.

Possible improvements

- Replace the CSV files with a database through JDBC (for example SQLite).
- Add a graphical interface (JavaFX) and automatic id generation.
- Add fines for overdue loans, renewals and a reservation queue.
- Write JUnit tests for `Library` so the rules are checked automatically.

B8 Conclusion

The mini-project met all eight functional requirements. The abstract class and interface let books and DVDs share code while differing in loan period, polymorphism let `Loan` compute due dates without knowing the item type, and encapsulation kept the borrowing rules from being bypassed. Custom checked exceptions turned rule violations into clear messages, and the CSV files kept the data between runs. Designing the class diagrams before finishing the code made the responsibilities of each class easy to separate, and the test table gave evidence that the rules work.

Part C: Overall Conclusion

Overall Conclusion

- The eleven labs covered the main ideas of object-oriented programming in Java in a sensible order. Labs 1 to 3 built the base: classes, references, constructors and encapsulation.
- Labs 4 to 6 added reuse and flexibility through inheritance, polymorphism and abstraction.
- Labs 7 to 10 added the tools needed for real programs: exceptions, generics and collections, enums and records, files, lambdas and streams.
- The mini-project then combined all of them in one working application.

Lessons Learned

- Object variables hold references, so assignment copies the reference and not the object.
- Hiding data behind validated methods makes invalid objects impossible, and returning copies protects internal state.
- Instance methods are chosen by the real object at run time, but fields and static methods follow the reference type.
- An `equals` override needs a matching `hashCode`, or hash-based collections misbehave.
- Checked exceptions make the compiler enforce error handling, and `finally` always runs when the `try` block is left.
- Choosing between inheritance and composition, and between an abstract class and an interface, is a design decision that should follow the is-a and can-do relationships.

Appendix B: Viva Questions &Answers

1. **Class vs object, and where each lives?**
   - A class is the blueprint (fields and methods);
   - An object is one concrete instance created with `new`.
   - The class definition is loaded once into the JVM's class-metadata area, along with its static members.
   - Objects live on the **heap**.
   - A variable like `Student s` holds only a reference to the object, and a local reference variable sits on the stack.
2. **Why no multiple inheritance of classes, and how do interfaces work around it?**
   - If a class extended two classes that both define the same method or field, Java could not tell which to inherit (the diamond problem), and the constructor chain would be unclear. A class may extend only one class but implement many interfaces, because interfaces hold no instance state.
3. **Overloading vs overriding?**
   - Overloading: same method name, different parameters, in the same class, chosen at compile time (`print(int)` and `print(String)`).
   - Overriding: a subclass supplies its own version with the same signature, chosen at run time (`Cat.sound()` replacing `Animal.sound()`).
   - Use `@Override` so a typo becomes a compile error.
4. **Superclass has no no-argument constructor and the subclass omits `super(...)`?**
   - Compile-time error. The compiler inserts `super();` automatically, and no matching constructor exists. Fix by calling `super(args)` as the first statement of the subclass constructor.
5. **Abstract class or interface?**
   - Choose an **abstract class** when related classes share state or common code, or you need constructors and protected members (Account with a balance and a shared `deposit`).
   - Choose an **interface** to describe a capability that unrelated classes can have, or when a class must have several such roles (Flyable, Comparable).
   - A class can extend only one abstract class but implement many interfaces.
6. **Why override `hashCode` whenever you override `equals`?**

- The contract says equal objects must have equal hash codes. Hash collections (HashMap, HashSet) use `hashCode` to pick a bucket and then `equals` to confirm.
- If only `equals` is overridden, two equal objects usually get different hash codes, land in different buckets, and the set stores duplicates or a map lookup fails.
- The reverse is not required: different objects may share a hash code.

7. **Checked vs unchecked exceptions?**
   - Checked exceptions extend `Exception` but not `RuntimeException`; the compiler forces you to catch or declare them. Examples: `IOException`, `FileNotFoundException`.
   - Unchecked exceptions extend `RuntimeException`; the compiler does not force handling, and they usually signal programming mistakes.
   - Examples: `NullPointerException`, `ArithmeticException` (also `ArrayIndexOutOfBoundsException`, `IllegalArgumentException`).
8. **Why no generics with primitives, and what is type erasure?**
   - Type parameters must be reference types, because after compilation `T` is replaced by `Object` (or its bound), and primitives are not objects.
   - Use wrappers (`List<Integer>`) and autoboxing.
   - **Type erasure** means the compiler checks the generic types, inserts casts, then removes the type arguments, so at run time `List<String>` and `List<Integer>` are the same class.
   - This is why you cannot write `new T()`, `instanceof List<String>`, or create a generic array.
   - It was done to stay compatible with pre-generics code.
9. **What does `final` mean?**
   - On a variable: it can be assigned only once (for a reference, the reference cannot change but the object's contents still can).
   - On a method: it cannot be overridden. On a class: it cannot be extended (`String` is final).
10. **Dynamic method dispatch; why are fields and static methods not polymorphic?**
    - With `Parent ref = new Child();`, a call to an overridden instance method runs the **Child** version, because the JVM picks the implementation from the actual object at run time. The compiler only uses the reference type to check that the method exists.
    - Fields, static methods and private methods are resolved at compile time from the reference type, so `ref.name` and `ref.stat()` give the Parent versions (they are hidden, not overridden).
11. **What is a functional interface, and how does a lambda relate?**
    - An interface with exactly one abstract method (it may also have default and static methods). `@FunctionalInterface` makes the compiler check it.

- A lambda is a short way to write the implementation of that single method, and the compiler works out which interface it targets from context.
- Examples: `Runnable`, `Comparator<T>`, `Predicate<T>`, `Function<T,R>`.

12. **How does try-with-resources differ from try/finally?**
    - Resources declared in the parentheses (anything implementing `AutoCloseable`) are closed automatically in reverse order, even if an exception is thrown.
    - With manual try/finally you must write the close code yourself and it is easy to get wrong.
    - If both the body and `close()` throw, try-with-resources keeps the original exception and attaches the other as a suppressed exception (`getSuppressed()`), whereas an exception from a `finally` block would replace the original.